CHAPTER

4 Undirected models

Bayesian networks used directed graphs to represent the semantics of a conditional probability expansion of the joint distribution. However, in some settings, it is more convenient to represent our models using an undirected semantic. Here we will examine two of the most common representations, **Markov random fields** and **factor graphs**. Both make use of a more generic factorization form for the joint distribution, which does not require that the factors correspond to conditional probabilities.

4.1 Factorization and Graphs

Suppose that $X = (X_1, \ldots, X_n)$ is a collection of random variables; we will typically assume these to be discrete, but will at times consider continuous random variables as well. An undirected model assumes that a function $f(X)$ factors into a product of non-negative **factors** f_α defined on (typically small) subsets of the variables X_α:

$$p(X) = \frac{1}{Z} f(X) \qquad f(X) = \prod_{\alpha \in \mathcal{I}} f_\alpha(X_\alpha) \qquad Z = \sum_x f(x)$$

Here, each $\alpha \subseteq [n] = \{1, \ldots, n\}$ is a subset of the variable indices, with X_α indicating the set of X_i with $i \in \alpha$. The set of variables X_α that are arguments to f_α are called its **scope**. The quantity Z is a constant that serves to normalize $p(X)$ and make it a valid probability distribution; for historical reasons it is usually called the *partition function*.

In a Markov random field, we construct a graph $G = (V, E)$ in which we associate a node $i \in V$ with each X_i, and add an edge $(i, j) \in E$ if X_i and X_j are both involved in some factor α, e.g., $\{i, j\} \subseteq \alpha$ for some $\alpha \in \mathcal{I}$. By construction, the sets α are **cliques** of G (fully connected sets of nodes).

A factor graph, on the other hand, is an undirected bipartite graph $G = (V \cup F, E)$, in which we again add a node $i \in V$ for each variable X_i (called the **variable nodes**), and in addition add a node $\alpha \in F$ for each factor f_α (called **factor nodes**). Then, we add an undirected edge (i, α) between variable node i and factor node α whenever $i \in \alpha$, i.e., X_i is an argument of factor f_α. In this text, we will draw variable nodes using circles, and factor nodes using squares.

In general, the factor graph depiction conveys more information about the factorization of f than the Markov random field graph. Figure 4.1 shows two factor graphs, each of which results in the same Markov graph. In particular, the factor graph in Figure 4.1(a) makes clear that the relationship between

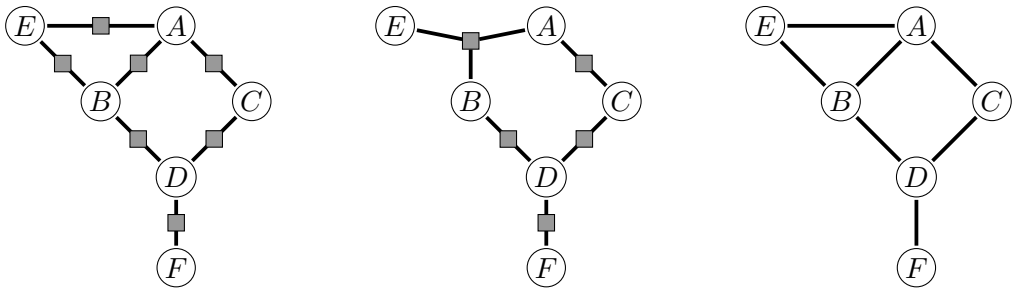


Figure 4.1: Often the factor graph representation of a function has more information than the Markov graph. For example, in (a) and (b) are two different factor graphs which both correspond to the same Markov graph, shown in (c). The graph in (a) makes clear that the relationship between A, B, E is comprised of three smaller functions, $f_{AB} \cdot f_{AE} \cdot f_{BE}$, while the graph in (b) has a single factor over all three variables, f_{ABE}. The graph (b) can represent any function that (a) can, by taking $f_{ABE} = f_{AB} \cdot f_{AE} \cdot f_{BE}$; however the converse is not true.

A, B, E is comprised of three smaller functions, $f_{AB} \cdot f_{AE} \cdot f_{BE}$, while the graph in (b) has a single factor over all three variables, f_{ABE}. The graph (b) can represent any function that (a) can, by taking $f_{ABE} = f_{AB} \cdot f_{AE} \cdot f_{BE}$; however the converse is not true. We say that the graph in (a) is **pairwise**, meaning that every factor f_α involves at most two variables, $\alpha = \{i, j\}$. In such cases, we may draw the Markov graph in (c) while specifying that it is a pairwise Markov random field, implying that we mean that f consists of exactly one factor per edge.

Finally, we say that a graphical model is **binary** if its random variables are all binary-valued, i.e., the X_i have domain size $|\mathcal{X}_i| = 2$ for all i. Logical or probabilistic logic languages often build some form of binary graphical model for reasoning (see Section 4.2).[1]

Connections to Physics?

Move to BM/IM example section?

- Boltzmann distribution? $\exp(-E(x)/kT)$
- Let energy $E(x) = \theta x$, for example
- Uncertainty in states varies with temperature: high temp, $f(x) \approx 1$ for all x; low temp, increasingly peaky (explain/example)
- Free energy; entropy; etc.

4.2 Examples of Undirected Graphs

There are many examples of special types of undirected graphical models; here we list a few of them.

[1]Note that this terminology is not entirely uniform across areas; for example in constraint satisfaction, "binary" typically refers to models with constraints over pairs of variables, i.e., what we term "pairwise" models.

Constraint networks

Constraint networks are a form of undirected model in which the factors correspond to *constraints* that forbid certain configurations of subsets of the variables. For example, the *graph coloring* problem asks for a color to be assigned to each region of a map, such that no two regions that share a border have the same color. We assign a variable to each region whose domain are the possible colors, and can encode the "not equal" constraints by taking

$$f_{ij}(X_i, X_j) = \begin{cases} 0.0 & \text{if } X_i = X_j \\ 1.0 & \text{otherwise} \end{cases}$$

for each pair (i, j) of neighboring regions. Then, the product

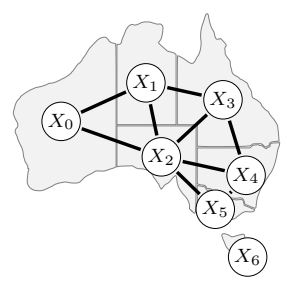
$$f(X) = \prod_{(i,j)\in E} f_{ij}(X_i, X_j) = \begin{cases} 0.0 & \text{if any constraint is violated} \\ 1.0 & \text{if all constraints are satisfied} \end{cases}$$

Then, we can see that finding the maximum value of $f(X)$ is equivalent to testing whether there exists some configuration x that satisfies all the constraints, while computing the partition function Z corresponds to counting the number of such configurations.

Example 4-1 : Map Coloring (R&N)

Suppose that we wish to color the map at right, using three colors $C = \{\text{red, green, blue}\}$. We assign each region a variable X_i taking on values in C, and connect variables associated with adjacent regions.

We can see that, taking $X = [X_0, \dots X_6]$, we have that $X = [r, g, b, r, g, r, b]$ is a solution, but $X = [r, g, b, r, g, b, r]$ is not, i.e.,



$$f([r, g, b, r, g, r, b]) = 1 \qquad \text{but} \qquad f([r, g, b, r, g, b, r]) = 0$$

since $f_{25}(X_2 = b, X_5 = b) = 0$.

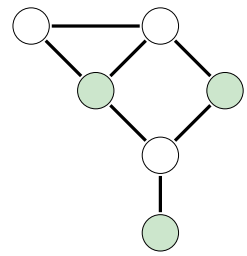
Another classic constraint satisfaction problem from graph theory is finding and counting independent sets in a graph. In this setting, we are given the graph G; then, an **independent set** is defined to be a collection of vertices of the graph, such that no two elements of the set are neighbors.

Example 4-2 : Independent Sets

Take X_i to be a binary-valued indicator of set membership, with $X_i = 1$ meaning that node i is in our set, and $X_i = 0$ if it is not. We define factors

$$f_{ij}(X_i, X_j) = 1 - \mathbb{1}[X_i = 1 \;\&\; X_j = 1]$$

for each edge in G, so that f_{ij} is zero (forbidding the configuration) if and only if both neighbors, X_i and X_j, are selected. Then, $p(X) \propto f(x) = \prod f_{ij}$ is a uniform distribution over all independent sets, and Z is the number of such sets.



A graph G and a maximum independent set (colored nodes).

To find the largest such independent set (a **maximum independent set**), whose size corresponds to the **independence number** of the graph, we can add factors

$$f_i(X_i) = a\mathbb{1}[X_i = 1] \qquad \text{where} \quad a > 1;$$

then $p(X)$ gives higher probability to larger independent sets, and any maximum independent set corresponds to a maximum of the function $f(X)$.

Many constraint problems are framed in terms of *propositional logic*. Suppose that we define a system comprised of Boolean (true/false) variables corresponding to possible events, and then express our problem in terms of a set of assertions of logical formulas on these variables, using logical connectives such as negation ($\neg$), conjuction ("and", $\wedge$), and disjuction ("or", $\vee$). This framework can allow our constraint problem to be expressed in a more readable form.

Example 4-3 : Logical Satisfaction (SAT)

We can define a set of logical atoms for the map coloring problem of Example 4-1 as $\{R_0, G_0, B_0, R_1, \ldots, B_6\}$, where R_i corresponds to the assertion that region i is colored red, G_i that it is colored green, etc. Then our knowledge can be written in terms of expressions on these variables:

$R_0 \vee G_0 \vee B_0$	Region 0 is colored blue, red, or green.
$R_0 \Rightarrow \neg R_1$	If Region 0 is red, Region 1 cannot be red.
$G_0 \Rightarrow \neg G_1$	If Region 0 is green, Region 1 cannot be green.
$\vdots$	$\vdots$

We can then attempt to prove or disprove statements about our system through constraint satisfaction; for example, to prove a statement S, we can add $\neg S$ to our knowledge base and show that the resulting system has no solution, i.e., no configuration can satisfy all the constraints (*proof by contradiction*).

Any propositional logic knowledge base can be reduced to a canonical form called *conjunctive normal form* (CNF), which consists of conjunctions ("ands") of formulas involving only disjunctions ("ors") of atoms (our original set of events or their negations). For example, the map coloring knowledge base might be written,

$$(R_0 \vee G_0 \vee B_0) \wedge (\neg R_1 \vee \neg R_0) \wedge (\neg G_1 \vee \neg G_0) \wedge \ldots$$

where each formula in parentheses is a disjunction of atoms. To view the overall CNF as a product of factors, $\prod_\alpha f_\alpha(\cdot)$, each disjunctive formula can be viewed as a factor $f_\alpha(\cdot)$ whose arguments are the atoms of that disjuction and whose value is zero for the configuration of arguments that evaluates to false.

While constraint networks can be used to encode deterministic relationships, in many systems it is useful to reason about probabilities. Again, these models may be expressed in a number of different styles, all exhibiting the same basic principles.

Boltzmann and Ising Models

Strictly positive distributions ($f(x) \neq 0$ for any x) over binary-valued X_i with only pairwise factors have two common representations, called Boltzmann machines (developed in computer science) and Ising models (developed in statistical physics). The **Boltzmann machine** model is defined using variables $X_i \in \{0, 1\}$, using a canonical exponential family form; we illustrate it using a small, three-variable example:

Example 4-4 : Boltzmann machine

Let $X_1, \ldots X_3$ be binary random variables, $X_i \in \{0, 1\}$. Define the distribution

$$\begin{aligned} p(X = x) \propto f(x) &= \exp\left[\theta_{12}x_1x_2 + \theta_{13}x_1x_3 + \theta_{23}x_2x_3 + \theta_1x_1 + \theta_2x_2 + \theta_3x_3\right] \\ &= f_{12}(x_1, x_2) \cdot f_{13}(x_1, x_3) \cdot f_{23}(x_2, x_3) \cdot f_1(x_1) \cdot f_2(x_2) \cdot f_3(x_3) \end{aligned}$$

The individual factors and joint model can be viewed as tables, e.g.,

x_1	f_1
0	1.0
1	$\exp(\theta_1)$

x_1x_2	f_{12}
0 0	1.0
0 1	1.0
1 0	1.0
1 1	$\exp(\theta_{12})$

$\cdots \Longrightarrow$

x_1 x_2 x_3	$f(x)$
0 0 0	1.0
0 0 1	$\exp(\theta_3)$
0 1 0	$\exp(\theta_2)$
0 1 1	$\exp(\theta_2 + \theta_3 + \theta_{23})$
1 0 0	$\exp(\theta_1)$
1 0 1	$\exp(\theta_1 + \theta_3 + \theta_{13})$
1 1 0	$\exp(\theta_1 + \theta_2 + \theta_{12})$
1 1 1	$\exp(\theta_1 + \theta_2 + \theta_3 + \theta_{12} + \theta_{13} + \theta_{23})$

We can interpret the parameters θ_i and θ_{ij} as expressing a "relative preference" for certain configurations of $X = (x_1, x_2, x_3)$. For example, if $\theta_1 > 0$, so that $\exp(\theta_1) > 1.0$, the factor $f_1(x_1)$ will increase the probability of configurations with $X_1 = 1$ relative to those with $X_1 = 0$. Unlike a Bayesian network, however, the parameters of a Boltzmann machine or other undirected model do not necessarily correspond to interpretable probabilities. For example, the actual marginal probability $\Pr[X_1 = 1]$ depends on all the parameters, and is given by summing the relevant entries of $f(x)$ and normalizing:

$$\Pr[X_1 = 1] = \Big(\sum_{x_2,x_3} f(X_1 = 1, x_2, x_3)\Big) \Big/ \Big(\sum_{x_1,x_2,x_3} f(x_1, x_2, x_3)\Big)$$

Notice that, if $\theta_{ij} = 0$, we have $f_{ij}(x_i, x_j) = 1$ for all x_i, x_j, a constant factor which does not influence the values of X. So, the undirected graph associated with a Boltzmann machine will connect each pair X_i, X_j where $\theta_{ij} \neq 0$.

It is often efficent to "vectorize" the representation, so that if x is a length-n vector of zeros and ones, then $p(X)$ can be equivalently written as

$$p(X = x) \propto f(x) = \exp\left[x^T Ax + b^T x\right],$$

where A is an upper-triangular matrix with entries $A_{ij} = \theta_{ij}$, $i \neq j$, $A_{i,i} = 0$, and b a length-n vector with entries $b_i = \theta_i$. If preferred, we can also substitute the symmetric matrix $\tilde{A} = \frac{1}{2}(A + A^T)$ for A instead.

Although the Boltzmann machine distribution is strictly positive for any finite values of the parameters θ, it is possible to express some constraint satisfaction problems as limits of Boltzmann models. For example, the independent set model of Example 4-2 can be viewed as the limiting distribution as we take $\theta_{ij} \to -\infty$ for each edge in G.

Another, practically equivalent representation for positive, binary pairwise distributions is the **Ising model** from phyics. Ising models were first described by Wilhelm Lenz (Lenz, 1920) and studied by his student Ernst Ising in his Ph.D. thesis (Ising, 1925) as a possible model for ferromagnetism in physics. The Ising model is an essentially equivalent model to the Boltzmann machine, but defined over binary *spin* variables $X_i \in \{-1, +1\}$ (sometimes called "up" or "down"), resulting in a different parameterization. To reprise Example 4-4:

Example 4-5 : Ising model

Let $X_1, \ldots, X_3$ be binary random variables with $X_i \in \{-1, +1\}$. Then, define:

$$\begin{aligned} p(X = x) \propto f(x) &= \exp\left[\theta_{12}x_1x_2 + \theta_{13}x_1x_3 + \theta_{23}x_2x_3 + \theta_1x_1 + \theta_2x_2 + \theta_3x_3\right] \\ &= f_{12}(x_1, x_2) \cdot f_{13}(x_1, x_3) \cdot f_{23}(x_2, x_3) \cdot f_1(x_1) \cdot f_2(x_2) \cdot f_3(x_3) \end{aligned}$$

The tabular representation of this model is:

x_1	f_1
-1	$\exp(-\theta_1)$
+1	$\exp(\theta_1)$

x_1	x_2	f_{12}
-1	-1	$\exp(\theta_{12})$
-1	+1	$\exp(-\theta_{12})$
+1	-1	$\exp(-\theta_{12})$
+1	+1	$\exp(\theta_{12})$

$\Rightarrow$

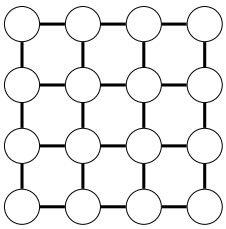
x_1	x_2	x_3	$f(x)$
-1	-1	-1	$\exp(-\theta_1 - \theta_2 - \theta_3 + \theta_{12} + \theta_{13} + \theta_{23})$
-1	-1	+1	$\exp(-\theta_1 - \theta_2 + \theta_3 + \theta_{12} - \theta_{13} - \theta_{23})$
-1	+1	-1	$\exp(-\theta_1 + \theta_2 - \theta_3 - \theta_{12} + \theta_{13} - \theta_{23})$
-1	+1	+1	$\exp(-\theta_1 + \theta_2 + \theta_3 - \theta_{12} - \theta_{13} + \theta_{23})$
+1	-1	-1	$\exp(+\theta_1 - \theta_2 - \theta_3 - \theta_{12} - \theta_{13} + \theta_{23})$
+1	-1	+1	$\exp(+\theta_1 - \theta_2 + \theta_3 - \theta_{12} + \theta_{13} - \theta_{23})$
+1	+1	-1	$\exp(+\theta_1 + \theta_2 - \theta_3 + \theta_{12} - \theta_{13} - \theta_{23})$
+1	+1	+1	$\exp(+\theta_1 + \theta_2 + \theta_3 + \theta_{12} + \theta_{13} + \theta_{23})$

Like Boltzmann machines, Ising models can be efficiently represented and evaluated using matrix-vector products. In the Ising model parameterization, the single-variable terms θ_i capture the preference of the X_i to take on value "up" (or positive, +1) or "down" (negative, -1), while the pairwise terms θ_{ij} capture their preference to take on the same value as their neighbors ($\theta_{ij} > 0$) or a different value ($\theta_{ij} < 0$). The Ising model has been well-studied on lattices in various dimensions and with various degrees of connectivity.

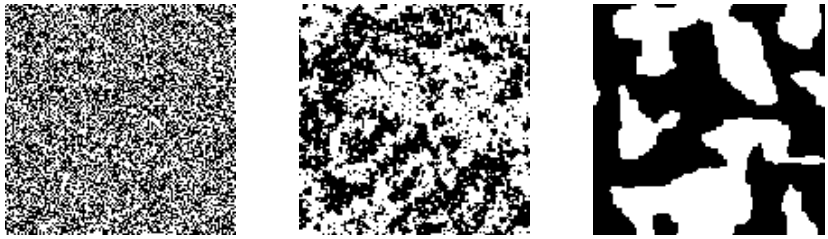
Example 4-6 : 2D Ising Grid

A two-dimensional lattice forms a grid, in which we can give each X_i a two-dimensional pixel location and connect it to its four neighbors, setting $\theta_{ij} = 0$ except on these pairs. Samples from the resulting distribution $p(X)$ behave intuitively: when $\theta_{ij} \approx 0$, the values of the X_i are nearly independent, and the image formed by X looks like white noise, while as θ_{ij} becomes large, we prefer for each pixel to be the same value, and the pixel colors become more and more spatially correlated:

4×4 Ising grid



Samples from $p(X)$, for a 128×128 grid:



Small θ_{ij} Moderate θ_{ij} Large θ_{ij}

While Boltzmann and Ising models are effectively equivalent, i.e., they can represent the same class of distributions, the different parameterizations can be more or less convenient in certain settings. For example, the Ising distribution form is useful for foreground-background segmentation problems in computer vision (e.g., Boykov and Jolly, 2001), in which the parameters θ_{ij} correspond to how strongly the model believes that two pixels or super-pixels should belong to the same class, so that θ_{ij} might be smaller across an edge in the image, or larger if the two regions are similar in color.

Probabilistic Logic

Just as propositional logic can be viewed as defining a constraint network, we can use also logic representations to provide a language for expressing probability distributions. In this setting, each clause $c(x_\alpha)$ may not define a constraint, but rather a preference for configurations in which that clause is true. (Equivalently, this preference can be viewed as the negative of a cost for violating the formula.) By assigning a preference θ_α to each clause, we can define a distribution that generalizes the Boltzmann form,

$$f(x) = \prod_\alpha \exp\big(\theta_\alpha \, c_\alpha(x_\alpha)\big),$$

i.e., all else being equal, configurations that satisfy $c_\alpha(x_\alpha)$ are more likely than those that do not, by a factor of $\exp(\theta_\alpha)$.

We also do not need to limit ourselves to propositional logic. Many languages for probabilistic logic, such as ProbLog (De Raedt and Kimmig, 2015) and Markov logic networks (Richardson and Domingos, 2006), allow the model to be expressed in a relatively simple form analagous to that of first order logic, defining logical variables and domains, and logical relationships that are assigned some probability or cost. Within such frameworks, it becomes easy to scale up the model by increasing the number of entities being reasoned about, or extend the model to include additional elements or sources of information.

Example 4-7 : Friends-Smokers network

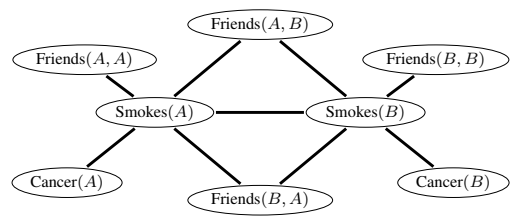
Let us consider a simple example of Markov logic from Richardson and Domingos (2006). We wish to reason about smoking and its effects in a social network, using a set of simple logical rules that we believe should (approximately) hold. We convert these rules into first-order logic, and assign a "weight" θ_α which indicates the strength of our belief that this rule should be true. So, for example, we might define the rules,

Smoking often causes cancer.
$\text{Smokes}(x) \Rightarrow \text{Cancer}(x)$ 1.5
Friends are more likely to have the same smoking habits.
$\text{Friends}(x, y) \Rightarrow \big(\text{Smokes}(x) \Leftrightarrow \text{Smokes}(y)\big)$ 1.1

Then, we can instantiate a model by allowing the logical variables x, y to range over a set of two entities, Alice (A) and Bob (B). This implicitly defines a set of Boolean events, Smokes(A), Smokes(B), Friends(A, A), Cancer(A), etc., and associated factors, for example:

S_A	C_A	$f(S_A, C_A)$
0	0	exp(1.5)
0	1	exp(1.5)
1	0	1.0
1	1	exp(1.5)

F_{AB}	S_A	S_B	$f(F_{AB}, S_A, S_B)$
0	0	0	exp(1.1)
0	0	1	exp(1.1)
0	1	0	exp(1.1)
0	1	1	exp(1.1)
1	0	0	exp(1.1)
1	0	1	1.0
1	1	0	1.0
1	1	1	exp(1.1)



Using these same relationships to define a model over, say, 10 friends becomes as simple as increasing the domain of x and y.

4.3 Conditioning and Conditional Independence

As with Bayesian networks, we would like to reason about the independence relationships among our variables directly from the graph representation, without resorting to computation on the actual probabilities. As it turns out, this is significantly simpler in undirected models than in Bayesian networks.

Conditioning as a Graph Operation

Let us first consider the operation of conditioning (fixing a known value for some random variable). It turns out the resulting function can also be represented using a graph, and that its graph is easy to obtain from that of the original function. This is most easily illustrated via an example:

Example 4-8 : Conditioning undirected models

Consider the Markov graph below, corresponding to the factorization

$$p(A, B, C, D, E) = \frac{1}{Z} f(A, B, C, D, E, F) = \frac{1}{Z}\, f_1(A, B, C)\, f_2(B, D, E, F).$$

If we observe the value $B = b$, we find that the joint probability with $B = b$ assigned is

$$p(A, B = b, C, D, E) = \frac{1}{Z} f(A, B = b, C, D, E) = \frac{1}{Z}\, f_1(A, B = b, C)\, f_2(B = b, D, E, F).$$

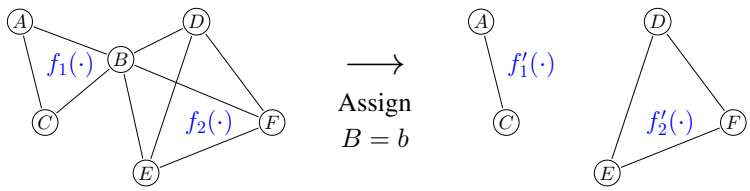
The function $f_1(A, B = b, C)$ is no longer a function of B (since B's value is known); if the variables are discrete, for example, it is a sub-table of the full factor f_1's table corresponding to only those entries where $B = b$. (See also, e.g., Example 2-6.) Denoting this new sub-table $f_1'(A, C)$, this means,

$$p(A, B = b, C, D, E, F) = \frac{1}{Z}\, f_1'(A, C)\, f_2'(D, E, F)$$

and,

$$p(A, C, D, E, F \mid B = b) = \frac{1}{Z'}\, f_1'(A, C)\, f_2'(D, E, F)$$

where Z' is the normalization constant for $f_1' \cdot f_2'$. Thus, graphically, the new conditional model has a Markov graph equivalent to that of f, but with the variable B and its incident edges removed:



$p(A, B, C, D, E, F)$ $\qquad$ $p(A, C, D, E, F \mid B = b)$

In general, the act of observing and fixing the value of some X_i removes the node X_i from the graph (since it is no longer a random variable, but rather a known value), as well as all the edges incident to X_i (since X_i is no longer a participant in the new, smaller factors). Thus, the factorization structure of the conditioned model can be easily derived directly from the graphical representation.

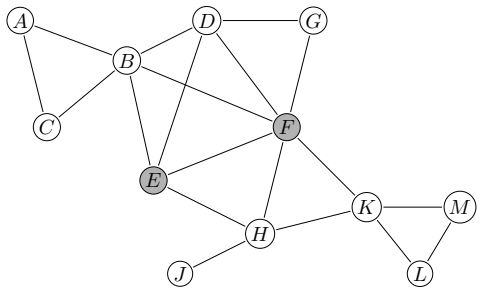
This relationship highlights an important difference between undirected networks, which represent collections of unnormalized factors, and directed Bayesian networks, which represent collections of normalized, conditional probability factors. In a Bayesian network, observing some child variable X_c created an unnormalized dependence between its parents. Although we created a simple rule (d-separation) for reasoning about independence given observations, the resulting conditional distribution was not necessarily easy to represent as a Bayesian network *without* variable X_c. In contrast, since factor graphs and Markov random fields do not assume that any factors are normalized, the unnormalized factors that result from the conditioning can be easily incorporated into the same graph representation as the original function. The simplicity of this graph transformation also implies a simpler rule for determining conditional independence in $p(X)$.

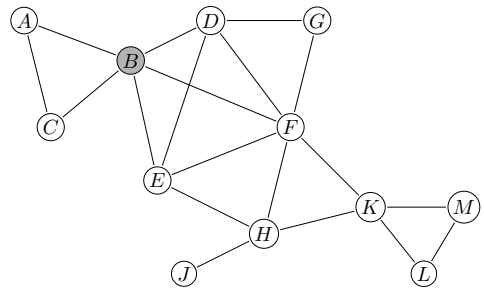
Evaluating Conditional Independence

Evaluating conditional independence is much simpler in undirected graphs than in Bayesian networks. As before, by inspecting only the graph structure, we are evaluating conditional independence relationships that *must* hold for *any* possible values of the factors f_α that comprise the model. Suppose we are given three sets of nodes, A, B, E in an undirected graph G. Then, we can conclude that X_A and X_B are independent given X_E if there every path in G between any $X_i, i \in A$ and $X_j, j \in B$ passes through some node $k \in E$. Conversely, if there is such a path, then there exists some set of factors f_α with undirected graph G in which X_A and X_B are not conditionally independent given X_E.

Example 4-9 : Conditional independence

Given an undirected graph, we can read conditional independence relations by checking separation in the graph. So, for example, in (a) we can see that the sets $\{A, B, C, D, G\}$ and $\{J, H, K, M, L\}$ are independent of each other given the values of $\{E, F\}$, since any path between the two sets must go through E or F. Similarly, in (b) we see that $\{A, C\}$ is independent of the rest of the variables given B.





We can see that any variable X_i will be independent of the rest of the graph given its immediate neighbors; this is called the Markov blanket of X_i. For example, in the above graph, the Markov blanket of E is $\{B, D, F, H\}$.

The intuition that the conditional model corresponds to a Markov graph in which the conditioned variable(s) have been removed makes it easy to see why undirected separation implies conditional independence. Suppose that, after conditioning, we have removed all paths between one set of nodes A and another set B. Grouping all the variables in A's connected component and denoting it A', and all the variables in B's connected component B', we know that the conditional model decomposes into a product of terms, some of which involve A', some involving B', but none involving both:

$$f(A', B') = f_1(A')\ f_2(B')$$

from which it is clear that A' and B' (and thus A and B) must be independent.

Hammersley-Clifford

To this point we have seen that, given a probability distribution $p(X)$ that can be written as a product of factors, we are able to draw a graph G such that separation in G implies conditional independence in $p(X)$; we say that $p(X)$ is *Markov* with respect to G or obeys *global Markov properties* with respect to G. But we may also ask a related question: suppose that we know that $p(X)$ is Markov with respect to G – can we then conclude that $p(X)$ factors in some way? As it turns out, the answer is yes, as long as $p(X)$ is strictly positive.

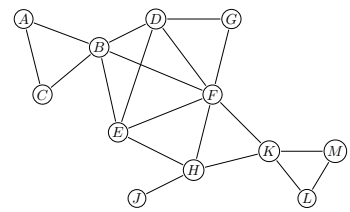
THEOREM 4.1 (Hammersley-Clifford)**.** *Suppose that a distribution $p(X)$ is strictly positive, $\forall x\ p(x) > 0$, and is Markov with respect to G. Then, $p(X)$ factors into a product,*

$$p(X) \propto \prod_{\alpha \in \mathcal{C}} f_\alpha(X_\alpha)$$

where $\mathcal{C}$ are the cliques *(fully connected subsets) of G.*

We can see this result mirrors our construction of the Markov random field graph G: given a factorization of $p(X)$, each term $f_\alpha(X_\alpha)$ defines edges between every pair of variables in its scope, creating a clique over the variables X_α. Conversely, the presence of a fully connected clique in G means that there may exist such a factor in the product.

Example 4-10 : Applying Hammersley-Clifford



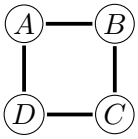
Consider the graph G shown at right; if a distribution $p(X)$ is strictly positive and Markov with respect to G, then it must admit a factorization of the form,

$$p([A, B, \ldots, M]) = \frac{1}{Z} f_1(A, B, C)\, f_2(B, D, E, F)\, f_3(D, F, G)$$
$$f_4(E, F, H)\, f_5(H, J)\, f_6(F, H, K)\, f_7(K, L, M)$$

with a factor for each maximal clique in G. Here, we have only included maximal cliques; any factors corresponding to non-maximal cliques (such as $\{B, D, F\}$) can be absorbed into the factor of any maximal clique that contains them ($\{B, D, E, F\}$).

However when some states are disallowed, so that $p(x) = 0$ for some x, it can be shown that $p(X)$ may satisfy the global Markov property while still not factoring into a product over the graph. Specifically, the following counterexample is due to Moussouris (1974):

Example 4-11 : Non-factoring Markov distribution



$ABCD$	$p(A, B, C, D)$
0000	0.125
0010	0.125
0100	0.000
0110	0.125
1000	0.000
1010	0.000
1100	0.000
1110	0.125
0001	0.125
0011	0.000
0101	0.000
0111	0.000
1001	0.125
1011	0.000
1101	0.125
1111	0.125

Suppose that we have a distribution $p(A, \ldots, D)$ and graph G given at right. It is easy to verify that the distribution satisfies the Markov relationships of the graph, since conditioning on any two opposite corners makes one of the other variables deterministic, and thus trivially independent of the last. For example, conditioning on $A = 0, C = 0$ implies that $B = 0$, while D takes value 0 or 1 with equal probability, so

$$p(B, D|A = 0, C = 0) = \text{Ber}(B\,;\,0) \cdot \text{Ber}(D\,;\,\tfrac{1}{2}),$$

where Ber is the Bernoulli distribution.

Now, assume that $p(\cdot)$ factors over the cliques (here, edges) of G. Then, examine each of the configurations with positive probability; we have for example,

$$p(0, 0, 0, 0) = f_{AB}(0, 0) \cdot f_{BC}(0, 0) \cdot f_{CD}(0, 0) \cdot f_{AD}(0, 0) = 0.125 > 0,$$

which implies that $f_{ij}(0, 0) > 0$ for each edge (i, j). Repeating for the other configurations, we find that all the f_{ij} are strictly positive; but this is a contradiction, since $p(\cdot)$ has some configurations with zero probabilities. So $p(\cdot)$ cannot be represented as a product over the cliques.

4.4 Converting between different model types

We have already seen many different flavors of graphical models, which share the same general principles but differ in the precise language in which those models are expressed. From a practical standpoint, different modeling libraries or software may be designed for specific types of models, such as Bayesian networks, pairwise Markov random fields, or factor graphs; some libraries may restrict models to logical (binary-valued) variables, while others do not. In this section we discuss to what degree, and how, we can convert between different styles of model. We first consider conversions between special types of undirected models, then discuss conversions between directed and undirected models.

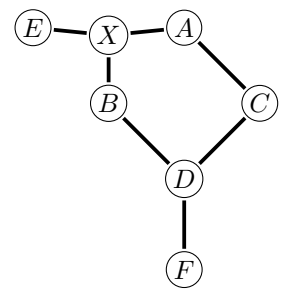
Converting between types of undirected models

Pairwise models

Pairwise undirected models are those in which each factor involves at most two variables. This is often convenient for drawing (illustrating) the graph structure, since there is no ambiguity in the factor scopes as in non-pairwise Markov random fields, and no need for multiple types of nodes as in factor graphs. It turns out that it is always possible to convert an undirected graphical model into a pairwise model, using a simple tranformation that may add additional variables to the model.

First, introduce additional nodes whose variables correspond to X_α, i.e., variables that take on values that are vectors of configurations of the variables in set α, for any non-pairwise factor f_α. Then, the factor f_α can be viewed as a function of a single variable (X_α), and we introduce pairwise constraint factors that enforce that the configuration of X_α is consistent with each of the configurations of the X_i, $i \in \alpha$.

Example 4-12 : Converting to pairwise models



Consider the factor graph from Figure 4.1(b), which had one non-pairwise factor over the three variables A, B, and E. We can create a new variable X whose domain consists of tuples of values (a, b, e), replace the factor $f_{ABE}(A, B, E)$ with $f_{ABE}(X)$, and add constraints that enforce consistency between X and A, B, and E, i.e.,

$$f_{AX}\big(A = a, X = (a', b', e')\big) = \begin{cases} 1 & a = a' \\ 0 & \text{otherwise} \end{cases}.$$

The resulting graph is pairwise, although it will not be strictly positive (some configurations are now inconsistent and have probability zero).

Binary models

A similar transformation, introducing additional variables, can be used to convert any undirected model over discrete variables into a binary-variable model, i.e., a model which consists of only Boolean variables. We define a variable for each possible value of each (non-binary) X_i or set X_α. For example, $X_{i;x} \in \{0, 1\}$ is one if $X_i = x$ and zero otherwise, i.e., $X_{i;x} = \mathbb{1}[X_i = x]$. This transformation is also closely related to the so-called *overcomplete* exponential family representation of a graphical model, which we discuss in more detail in Section 4.5. For an explicit example of this binary-variable transformation, see Example 4-3, which corresponds to a binary version of the three-state map coloring problem of Example 4-1. While this conversion method introduces non-positive factors, i.e., factors that exclude certain configurations by assigning them probability zero, it is also possible to design a (somewhat more complex) model transformation that preserves strict positivity; see Eaton and Ghahramani (2013) for details.

However, while any model can be made binary, or can be made pairwise, not every model can be made binary and pairwise. In particular, models with non-positive factors (such as the output of our simple binary transformation) are difficult to make pairwise, since for example, the constraint that $X_{i;x} = 1$ for some value of x (i.e., that X_i takes on exactly one of its possible values) is not easy to encode directly in a pairwise model. However, Eaton and Ghahramani (2013) showed that one can express such "one-hot"

factors as a limit of a collection of pairwise factors over the $X_{i;x}$ and a set of additional, auxilliary binary variables, demonstrating that for any discrete graphical model $p(X)$ there exists a binary, pairwise model that can be made arbitrarily close to p.

Converting between directed and undirected models

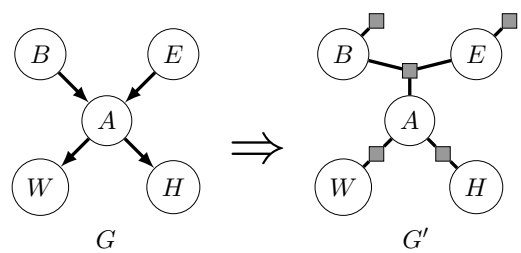
Directed and undirected graphical models have different strengths. Directed models are often more appropriate for models in which we are trying to capture a notion of "cause": the value of parent variables cause changes in the distribution of their children, and so forth. In contrast, undirected models can be more appropriate when trying to capture a sense of "correlation", i.e., that patterns of values tend to appear together (for example, a pixel taking the same value as its neighboring pixel) without imposing a causal order.

However, we may want to convert a directed model to an undirected, or vice versa, for a number of reasons. For example, the software or programming language in which we represent our model may be tailored to use either directed models or undirected models, and we must convert our model to fit the software's framework.

Directed to Undirected

Representing a Bayesian network as an undirected graphical model is relatively easy. The Bayesian network represents a factorization into conditional probability distributions, $p(X_i|X_{\text{pa}(i)})$; representing each of these as factors in a Markov random field means connecting each pair of variables in $X_{i\cup\text{pa}(i)}$, i.e., connecting X_i and all of its parents. This process is sometimes called **moralizing** the graph, since it joins together any pair of variables that share a child.

Example 4-13 : Burglar Alarm Factor Graph



We can easily represent the burglar alarm Bayes net graph G as a factor graph, by simply introducing one factor per conditional probability table. Since the conditional probability table for each X_i is a function of X_i and its parents, $X_{\text{pa}(i)}$, this factor connects to X_i and its parent set.

In the Markov graph, this means these variables will all be connected. So, the Markov graph includes all the edges of G (now undirected), plus edges connecting each pair of nodes in the parent sets $X_{\text{pa}(i)}$ – here, just the pair B, E, which were unjoined parents of A.

While this procedure is straightforward, it does lose some of the conditional independence information available from the Bayes net. For example, we can tell from the directed graph that the parents of A are independent if none of A and its descendants are observed; in the undirected graph representation, this is not clear, since E and B are directly joined by an edge. Mathematically, this can be understood since this independence arises from the normalization of $p(A|E, B)$; if A is marginalized over, $\sum_A p(A|E, B) = 1$ for all E, B. However, this independence does *not* hold for any function over A, B, E, and thus the undirected graph (which does not know anything about the values of the factors it represents) cannot conclude that B and E are independent.

Undirected to Directed

Representing an undirected graph as a Bayes net is slightly more complicated. There are two basic approaches we can use.

Our first option is to construct a Bayes net over the variables X_i, using the undirected graph to determine which dependencies are required. First, we select an order over the X_i; as in Chapter 3, this order is important to obtaining a compact Bayes net, but the optimal order may be hard to determine. Then, we proceed along this order, and for each X_i we identify the minimal set of *earlier* variables that will make X_i independent from all other earlier variables as X_i's parents – in other words, $X_{\text{pa}(i)}$ include enough of the earlier variables that there is no path from any node earlier than X_i to X_i that does not pass through $X_{\text{pa}(i)}$. However, as with the conversion from directed to undirected, this procedure may lose some of the useful independence information that was present in the original graph.

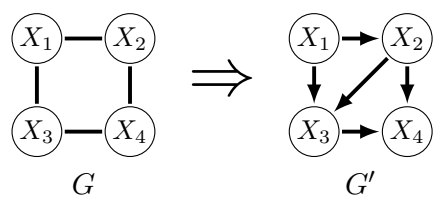
Example 4-14 : Undirected to Directed Graphs (1)

Consider a probability distribution that we know is Markov with respect to a 2×2 undirected grid. As in Chapter 3 we can select an ordering to the variables, say, $[X_1, X_2, X_3, X_4]$, and create a Bayes net by checking each term in the chain rule expansion for any conditional independence in G. So:

- Since X_1 is first, it is a root
- X_2 is not independent of X_1 in G, so we add an edge $X_1 \to X_2$.
- X_3 is not independent of X_1 given X_2, nor of X_2 given only X_1, so we add edges from both.
- X_4 is not independent of X_3 or X_2, but is conditionally independent of X_1 given X_2, X_3.

This results in the directed graph G' at right.

Notice that some information has been lost; from the directed graph, we cannot see via d-separation that X_2 and X_3 are independent given X_1, X_4 (which is clear from the undirected graph). All the conditional independencies of the directed graph also hold in the original undirected G, but there may be additional independencies in G that we cannot see from only the structure of G'.



Another option is to create a number of extra variables and treat them as observed. Specifically, for each factor f_α in the undirected graph, we create a variable $Z_\alpha \in \{0, 1\}$, and define its conditional distribution to be

$$p(Z_\alpha | X_\alpha) = \begin{cases} f_\alpha(X_\alpha)/f^*_\alpha & \text{if } Z_\alpha = 0 \\ 1 - f_\alpha(X_\alpha)/f^*_\alpha & \text{if } Z_\alpha = 1 \end{cases} \qquad f^*_\alpha = \max_{x_\alpha} f(x_\alpha)$$

$$p(X_i) = \frac{1}{|\mathcal{X}_i|} \qquad \text{(uniform)}$$

The quantity f^*_α ensures these probabilities are between zero and one; by observing "evidence" $Z_\alpha = 0$, we produce a Bayes net that matches our undirected distribution up to a constant:

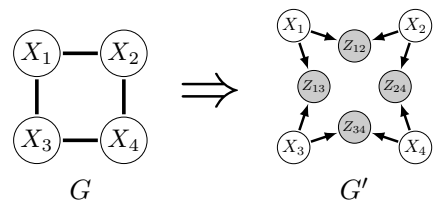
$$\begin{aligned} \prod_\alpha f_\alpha(X_\alpha) &= \left(\prod_i |\mathcal{X}_i|\right) \left(\prod_\alpha f^*_\alpha\right) \prod_\alpha p(Z_\alpha = 0 | X_\alpha) \prod_i p(X_i) \\ &= (\text{const}) \cdot p(X, Z = 0) \end{aligned}$$

Other than the nuisance factor of the constant (required to ensure that $p(X, Z)$ is a probability), this Bayes net is exactly equivalent to the original undirected model. However, it loses many of the details that make Bayesian networks intuitive – it now includes many variables that do not have an intuitive meaning in the problem, and we cannot derive any "causal" intuition from the directionality of the graph.

Example 4-15 : Undirected to Directed Graphs (2)

Consider a positive probability distribution that is Markov with respect to a 2×2 undirected grid G. This means that $p(X)$ must factor into terms $f_{12}(X_1, X_2), \ldots, f_{34}(X_3, X_4)$.

For concreteness, assume X_1, X_2 are binary, and that f_{12} is given by the table below. Dividing by the largest entry ($f_{12}([1, 1]) = 10.0$) gives all values between zero and one, and we can define $p(Z|X_1, X_2)$ as,



x_1 x_2	$f_{12}(x)$
0 0	3.0
0 1	1.0
1 0	5.0
1 1	10.0

$\Longrightarrow$

$p(Z_{12}\|X_1, X_2)$	$Z_{12} = 0$	$Z_{12} = 1$
$X_1, X_2 = 00$	0.3	0.7
$X_1, X_2 = 01$	0.1	0.9
$X_1, X_2 = 10$	0.5	0.5
$X_1, X_2 = 11$	1.0	0.0

After artificially "observing" $Z_{12} = 0$, the new Bayes net factor will be equivalent to f_{12} up to a scaling constant:

$$f_{12}(X_1, X_2) = 10\, p(Z_{12} = 0|X_1, X_2).$$

Repeating for each factor f_{ij} gives the directed graph G'.

Although this approach does precisely capture the independence properties of the original graph, its topology does not offer any insight into the causal relationships of the model.

4.5 The overcomplete exponential family

We saw in Chapter 2 that many distributions of interest could be expressed as members of the exponential family. Suppose that we define a positive distribution over a collection of discrete-valued X_i using the product of factors,

$$p(X) \propto f(X) = \prod_{\alpha} f_\alpha(X_\alpha).$$

Since the X_i are discrete, the factors f_α can be viewed as tables; if we parameterize these tables in the standard way (one parameter per table entry), we find that the resulting distribution is an example of an exponential family distribution called the **overcomplete exponential family**. Specifically,

$$f_\alpha(X_\alpha) = \exp\Big(\sum_{x'_\alpha \in \mathcal{X}_\alpha} \theta_{x'_\alpha} \mathbb{1}[X_\alpha = x'_\alpha] \Big)$$

where we have one parameter θ_k per joint configuration $k = x'_\alpha$ of the variables X_α, as well as one feature $\phi_k(X_\alpha) = \mathbb{1}[X_\alpha = x'_\alpha]$. This construction is analagous to the two-parameter Bernoulli representation in Example 2-11, and also to the graph transformation to binary-only variables described in Section 4.4.

Example 4-16 : Overcomplete exponential family

Suppose we have $f(X) = f_{12}(X_1, X_2)\, f_{13}(X_1, X_3)\, f_{23}(X_2, X_3)$, with $X_i \in \{0, 1, 2\}$ and pairwise factors given by the tables:

f_{12}	$X_1 =$ 0	1	2
$X_2 =$ 0	0.7	2.5	3.6
1	3.0	2.1	0.4
2	1.8	0.5	3.8

f_{13}	$X_1 =$ 0	1	2
$X_3 =$ 0	1.5	0.5	0.4
1	0.8	0.9	0.4
2	2.5	2.2	2.1

f_{23}	$X_2 =$ 0	1	2
$X_3 =$ 0	4.4	3.9	3.8
1	0.3	2.3	4.6
2	4.1	3.4	2.7

Let us define the function $\theta(X) = \log f(X)$; then we can express $\theta(X)$ as,

$$\begin{aligned}\theta(X) = \log f(X) = \log(0.7)\mathbb{1}[[X_1, X_2] = 00] + \log(3.0)\mathbb{1}[[X_1, X_2] = 01] + \dots \\ + \log(1.5)\mathbb{1}[[X_1, X_3] = 00] + \dots \\ + \log(4.4)\mathbb{1}[[X_2, X_3] = 00] + \dots\end{aligned}$$

The function $\theta(X)$ is linear, and can be expressed as a vector product, $\theta(X) = \theta \cdot \phi$, where θ is a vector of parameters, and ϕ a vector of features of X:

$$\begin{aligned}\theta &= \begin{bmatrix} \log(0.7) & \dots & \log(1.5) & \dots & \log(2.7) \end{bmatrix} \\ \phi(X) &= \begin{bmatrix} \mathbb{1}[X_{12} = 00] & \dots & \mathbb{1}[X_{13} = 00] & \dots & \mathbb{1}[X_{23} = 22] \end{bmatrix}\end{aligned}$$

This vector representation puts $p(X) \propto f(X) = \exp\big(\theta(X)\big) = \exp\big(\theta \cdot \phi(X)\big)$ into canonical exponential family form.

The resulting model is called *overcomplete* because its features are not linearly independent of one another. In Example 4-16, we can see that

$$\begin{aligned}\mathbb{1}[X_{12} = 00] + \mathbb{1}[X_{12} = 01] + \mathbb{1}[X_{12} = 02] &= \mathbb{1}[X_1 = 0] \\ &= \mathbb{1}[X_{13} = 00] + \mathbb{1}[X_{13} = 01] + \mathbb{1}[X_{13} = 02]\end{aligned}$$

since X_2 and X_3 must take one one of the three values. The features' linear dependence means that the parameter values are not uniquely determined by the model probabilities, i.e., they can be changed without changing the resulting probability distribution: if we, say, add a scalar value v to the parameters of each feature $\mathbb{1}[X_{12} = 0x]$, and subtract v from the parameters of each feature $\mathbb{1}[X_{13} = 0x]$, the two changes will cancel with one another and result in the same probability distribution over X. We call changes to the parameter values that leave the underlying distribution unchanged **reparameterizations** of the model.

4.6 Gaussian models

The multivariate Gaussian distribution is an exponential family model that can be interpreted as a graphical model. The density function is given by,

$$p(X) = \mathcal{N}(X; \mu, \Sigma) = (2\pi)^{-\frac{n}{2}} |\Sigma|^{-\frac{1}{2}} \exp\Big(-\frac{1}{2}(X - \mu)\Sigma^{-1}(X - \mu)^T\Big)$$

where μ and Σ are the mean and covariance matrix that define the center and shape of the Gaussian probability. This representation is sometimes called the "moment" form of the Gaussian, since it is expressed in terms of first moments ($\mu_i = \mathbb{E}[X_i]$) and second moments ($\Sigma_{ij} = \mathbb{E}[X_i X_j] - \mu_i \mu_j$).

Rearranging, we can see that this distribution is in the exponential family, with canonical parameters

$$J = \Sigma^{-1} \qquad\qquad h = \mu\Sigma^{-1}$$

The re-parameterized distribution form is often called the "information" form of the Gaussian; J is called the precision matrix, and h the information mean. To distinguish this parameterization, we write

$$p(X) = \mathcal{N}^{-1}(X; h, J) = (2\pi)^{-\frac{n}{2}} |J|^{\frac{1}{2}} \exp\Big(-\frac{1}{2} X J X^T + hX - \frac{1}{2} h J^{-1} h \Big)$$

From this representation, we can see that $\mathcal{N}^{-1}(X; h, J)$ is a pairwise graphical model (since it involves terms $h_i X_i$ and $J_{ij} X_i X_j$), with an edge connecting any two variables for which $J_{ij} \neq 0$. Thus, sparsity in the graph (few edges) precisely corresponds to sparsity in the inverse covariance J.

Interestingly, these two representations are useful for different queries about the model. For example, in the classical "moment" form, it is easy to marginalize over some variables to produce a distribution over the remainder

$$p(X_A, X_B) = \mathcal{N}(X = [X_A, X_B]; \mu, \Sigma) \qquad \Rightarrow \qquad p(X_A) = \mathcal{N}(X_A\,;\, \mu_A, \Sigma_{AA})$$

where μ_A and Σ_{AA} are the sub-vector and sub-matrix associated with the indices A.

On the other hand, in the information form, it is easy to *condition* on observations of some variables:

$$\begin{aligned} & p(X_A, X_B) = \mathcal{N}(X = [X_A, X_B]; \mu, \Sigma) = \mathcal{N}^{-1}(X = [X_A, X_B]\,;\, h, J) \\ \Rightarrow \qquad & p(X_A | X_B = x_B) = \mathcal{N}^{-1}(X_A\,;\, h_A - J_{AB} \cdot x_B, J_{AA}) \end{aligned}$$

where J_{AA} is the sub-matrix of J associated with the indices A. As with the graphical viewpoint of the conditioning operation, we see that we are left with a graph defined by the non-zero entries of J_{AA}, i.e., the edges in the graph of J that do not involve any of the variables in B.

Example 4-17 : Gaussian Markov Chain

Consider a simple Markov chain formed by Gaussian random variables, say:

$$x_1 \sim \mathcal{N}(X_1\,;\, 0, 1) \qquad x_2 \sim \mathcal{N}(X_2\,;\, \frac{1}{2} x_1, 0.5) \qquad x_3 \sim \mathcal{N}(X_3\,;\, x_2, 1)$$

in which X_2 depends on X_1, and X_3 on X_2 but not on X_1. It is easy to see that the distribution is zero mean, since $\mathbb{E}[X_3] = \mathbb{E}[X_2] = \frac{1}{2}\mathbb{E}[X_1] = 0$, and we can similarly show that the entries of the covariance matrix, $\Sigma_{ij} = \mathbb{E}[X_i X_j]$ are given by:

$$\begin{aligned} & \mathbb{E}[X_1^2] = 1 && \mathbb{E}[X_1 X_2] = \mathbb{E}[X_1 \cdot \tfrac{1}{2} X_1] = \tfrac{1}{2} && \mathbb{E}[X_1 X_3] = \mathbb{E}[X_1 X_2] = \tfrac{1}{2} \\ & \mathbb{E}[X_2^2] = \mathbb{E}[\tfrac{1}{4} X_1^2] + \tfrac{1}{2} = \tfrac{3}{4} && \mathbb{E}[X_2 X_3] = \mathbb{E}[X_2^2] = \tfrac{3}{4} && \mathbb{E}[X_3^2] = \mathbb{E}[X_2^2] + 1 = \tfrac{7}{4} \end{aligned}$$

Converting to information form, we see that

$$\Sigma = \begin{bmatrix} 1 & 0.5 & 0.5 \\ 0.5 & 0.75 & 0.75 \\ 0.5 & 0.75 & 1.75 \end{bmatrix} \qquad \Rightarrow \qquad J = \Sigma^{-1} = \begin{bmatrix} 1.5 & -1 & 0 \\ -1 & 3 & -1 \\ 0 & -1 & 1 \end{bmatrix}$$

showing that the non-zero pattern of J matches the adjacency matrix of the graph: $J_{13} = 0$, indicating no edge between X_1 and X_3, and so X_1 and X_3 are conditionally independent given X_2.

4.7 Learning from data

So far, we have seeen how undirected graphs allow us to compactly represent functions defined over large collections of variables. Although some undirected models (such as constraint satisfaction problems) are typically defined *a priori*, in many settings we may want to *learn* the model, i.e., fit the parameters of our model to observed data. The fact that our model parameters define an *un-normalized* probability distribution makes parameter fitting somewhat more complicated than in directed Bayesian networks, however.

Maximum likelihood learning

The standard technique for fitting a probability model to data is to optimize the model parameters to maximize the data likelihood, as discussed in Section 2.3. Many undirected graphical models, such as discrete models defined by table-based representations (Section 4.5) and Gaussian distributions (Section 4.6), are equivalent to exponential family distributions in canonical form, whose features are defined by the graph structure. Thus, standard results for exponential family distributions can be directly applied: the log-likelihood function is convex (see Example 2-23), and that its gradient is given by Eq. (2.3) in Section 2.3.

For concreteness, consider a table-based undirected model whose parameters consist of all the entries of the log-factors, $\theta_\alpha(X_\alpha)$, as in Example 4-16. Then, the log-likelihood is,

$$\mathcal{L} = \sum_s \sum_\alpha \theta_\alpha(x^{(s)}) - m \log Z(\theta).$$

The first term is easy to evaluate (by looking up each data point's values in our tables), but the log partition function may not be, as it (nominally, at least) requires a sum over all the exponentially many possible configurations of the full state vector x. Similarly, if we evaluate the gradient of $\mathcal{L}$ in order to optimize over θ, we find that

$$\frac{\partial \mathcal{L}}{\partial \theta_\alpha(x_\alpha)} = \sum_s \mathbb{1}[x_\alpha^{(s)} = x_\alpha] - m\mathbb{E}\big[\mathbb{1}[X_\alpha = x_\alpha]\big],$$

i.e., for gradient ascent we should adjust the entry for partial configuration x_α in each table $\theta_\alpha(\cdot)$ by the difference between the empirical probability of seeing configuration x_α in the data and the model's current probability of that configuration.

Compared to the simple closed-form parameter estimates for a Bayesian network, training and evaluating undirected models is complicated by the need to compute the log-partition function $\log Z$ and/or expectations of the features, $\mathbb{E}[\phi]$ during optimization. These computations are fundamentally difficult in general; we study the computational complexity of these tasks in Chapter 5. In practice, many model fitting techniques replace these computations with approximate estimates of the sum or expectations, which are the focus of several later chapters.

Example 4-18

Expand. Learning a basic Bernoulli model, $p(X) = \frac{1}{Z}\exp(\theta X)$.

$$\mathcal{L}(\mathcal{D}) = \sum_{\int} \log\{(\S^{(\int)}) - \Updownarrow \log \mathcal{Z} = \theta(\sum_{\S} \S^{(\int)}) \; - \; \Updownarrow \log \mathcal{Z}.$$

So, the first term is linear in θ; it is the second term $\log Z$ that is strictly convex, and results in the uniqueness of the maximum likelihood estimate, which balances the value of the two terms. **Plots.**

Tree-structured graphs

If our Markov random field model is a tree, i.e., the graph is pairwise and acyclic, we can leverage the equivalence between the MRF's conditional independence relationships and those of its associated Bayesian network representation to estimate the maximum likelihood parameters.

In Section 3.3 we saw that we could easily compute the maximum likelihood estimates of the conditional probabilities for a Bayesian network in closed form, using the empirical statistics of the data. Compared to the undirected case, this can be viewed as a consequence of the fact that the conditional probability parameterization of the Bayesian network automatically enforces that $Z(\theta) = 1$, i.e., that the distribution is normalized.

However, if our undirected model is equivalent to (induces the same conditional independence relationships as) some Bayesian network, we can leverage this fact to estimate parameter values that maximize the likelihood of the undirected model. In particular, if our Markov random field model is a tree, i.e., the graph is pairwise and acyclic, then there exists a Bayesian network model that is equivalent. We can select any node of the undirected graph as the root, and then orient the direction of our edges away from the root to obtain an equivalent Bayesian network. Finally, we estimate the conditional probabilities for each edge (i, j), and these parameters can be selected as the values of the tables $\theta_{ij}(x_i, x_j)$.

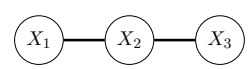
Since the tabular factor form is overcomplete, there can be many possible settings for the θ_{ij} that correspond to the same joint probability distribution. For example, any topological order of the tree gives a different set of conditional probabilities (and thus different tables over the pairs of variables), but the same overall joint distribution. For an undirected tree-structured model, it can sometimes be convenient to use a more symmetric estimator of the tables, namely,

$$\hat{p}(X) = \Big(\prod_i \hat{p}_i(X_i)\Big)\Big(\prod_{ij\in E} \frac{\hat{p}_{ij}(X_i, X_j)}{\hat{p}_i(X_i)\,\hat{p}_j(X_j)}\Big) \tag{4.1}$$

where $\hat{p}_i$ and $\hat{p}_{ij}$ are the single-variable and pairwise empirical marginals estimated from the data. An example illustrates this point:

Example 4-19 : Maximum likelihood in trees

Consider fitting a simple Markov tree graph using the same data set as in Example 3-7, shown at right. The independence assumptions of the Markov graph are identical to those of the Bayesian network in Example 3-7, $X_1 \to X_2 \to X_3$. So, the parameter estimates computed in that example also correspond to a maximum likelihood estimate of the undirected model.



Markov Graph

X_1	X_2	X_3
1	0	0
1	1	1
1	1	1
1	0	1
1	1	1
1	1	0
0	1	0
1	1	1
1	0	0
0	0	1

Observed dataset

We can see that, in fact, many Bayesian networks are equivalent to the same undirected model; for example, $X_1 \leftarrow X_2 \leftarrow X_3$, or $X_1 \leftarrow X_2 \to X_3$ (but not the poly-tree, $X_1 \to X_2 \leftarrow X_3$!). Finding maximum likelihood estimates (using the empirical conditional probabilities) for any of these equivalent models will produce an identical distribution, only with differently parameterized factors; for example,

$$\begin{aligned}\hat{p}(X = [000]) &= \hat{p}_3(X_3 = 0)\,\hat{p}_{2|3}(X_2 = 0|X_3 = 0)\,\hat{p}_{1|2}(X_1 = 0|X_2 = 0)\\ &= (4/10) \cdot (2/4) \cdot (1/4)\\ &= 0.05,\end{aligned}$$

the same probability that we computed using the empirical estimates of $p_1(X_1)\,p_{2|1}(X_2|X_1)\,p_{3|2}(X_3|X_2)$. Similarly, the symmetric form (4.1) gives,

Algorithm 4.1 Iterative proportional fitting for graphical models

Input: Graphical model $p(X) \propto \prod_{\alpha \in \mathcal{I}} f(X_\alpha)$; data set D and/or empirical marginals $\hat{p}_D(X_\alpha)$.

Output: Maximum likelihood estimate of $p(X)$.

repeat
  for $\alpha \in \mathcal{I}$ **do**
    Marginalize to find $p(X_\alpha)$
    Update $f_\alpha(X_\alpha) \leftarrow f_\alpha(X_\alpha) \cdot \big(\hat{p}_D(X_\alpha)/p(X_\alpha)\big)$
  end for
until converged

The factors f_α are typically normalized in some way for numerical stability after each update.

$p(X_1)$	$X_1 = 0$	$X_1 = 1$
	$2/10$	$8/10$

$p(X_2)$	$X_2 = 0$	$X_2 = 1$
	$4/10$	$6/10$

$p(X_3)$	$X_3 = 0$	$X_3 = 1$
	$4/10$	$6/10$

$\frac{p(X_1,X_2)}{p(X_1)p(X_2)}$	$X_2 = 0$	$X_2 = 1$
$X_1 = 0$	$\frac{(1/10)}{(2/10)(4/10)}$	$\frac{(1/10)}{(2/10)(6/10)}$
$X_1 = 1$	$\frac{(3/10)}{(8/10)(4/10)}$	$\frac{(5/10)}{(8/10)(6/10)}$

$\frac{p(X_2,X_3)}{p(X_2)p(X_3)}$	$X_3 = 0$	$X_3 = 1$
$X_2 = 0$	$\frac{(2/10)}{(4/10)(4/10)}$	$\frac{(2/10)}{(4/10)(6/10)}$
$X_2 = 1$	$\frac{(2/10)}{(4/10)(6/10)}$	$\frac{(4/10)}{(6/10)(6/10)}$

so that

$$\begin{aligned}
\hat{p}(X = [000]) &= \hat{p}_1(X_1 = 0)\, \hat{p}_2(X_2 = 0)\, \hat{p}_3(X_3 = 0)\, \frac{\hat{p}_{12}(X_{12} = 00)}{\hat{p}_1(X_1 = 0)\hat{p}_2(X_2 = 0)}\, \frac{\hat{p}_{23}(X_{23} = 00)}{\hat{p}_1(X_2 = 0)\hat{p}_2(X_3 = 0)} \\
&= (2/10) \cdot (4/10) \cdot (4/10) \cdot \frac{(1/10)}{(2/10)(4/10)} \cdot \frac{(2/10)}{(4/10)(4/10)} \\
&= 0.05
\end{aligned}$$

Iterative proportional fitting

In general, for graphical models with cycles, the complex relationship between the model parameters as a whole and the normalization constant $\log Z$ stymies our ability to solve for the maximum likelihood parameter settings in closed form. However, the zero-gradient condition defined by matching the model and data expectations can still be useful in optimizing the parameter values.

Consider a block coordinate ascent procedure, in which we iteratively update small subsets of the parameters while holding the others fixed. If we update only the parameters involved with a single log-factor θ_α, we can often maximize the likelihood with respect to that subset of parameters in closed form.

Again for concretness, let us assume a table-based parameterization of a discrete model, and suppose we have computed the expectations associated with factor $\theta_\alpha(X_\alpha)$:

$$\begin{aligned} p(x_\alpha) &= \mathbb{E}\big[\mathbb{1}[X_\alpha = x_\alpha]\big] \qquad \forall x_\alpha \in \mathcal{X}_\alpha \\ &= Z^{-1} \exp\big(\theta_\alpha(x_\alpha)\big) \sum_{x \sim x_\alpha} \exp\big(\sum_{\beta \neq \alpha} \theta_\beta(x_\beta)\big) \\ &\propto \exp\big(\theta_\alpha(x_\alpha)\big)\, g(x_\alpha) \end{aligned}$$

where we have explicitly pulled out the factor $f_\alpha = \exp(\theta_\alpha)$ to show that the probability table $p(X_\alpha)$ involves a complex sum over the rest of the model, $X \setminus X_\alpha$, multiplied by f_α and then normalized. Holding the rest of the model (and thus $g(\cdot)$) fixed, we have that the desired (empirical) marginal probabilities $\hat{p}$ can be expressed as,

$$\hat{p}(X_\alpha) = \frac{\hat{p}(X_\alpha)}{p(X_\alpha)} p(X_\alpha) = \frac{\hat{p}(X_\alpha)}{p(X_\alpha)} f_\alpha(X_\alpha)\, g(X_\alpha)$$

and therefore the update

$$f_\alpha(X_\alpha) \leftarrow \frac{\hat{p}(X_\alpha)}{p(X_\alpha)} f_\alpha(X_\alpha)$$

will ensure that the model's marginal $p(X_\alpha)$ matches the empirical marginals $\hat{p}(X_\alpha)$. Note that f_α can also be rescaled by a constant (since any constant factor will simply be absorbed into the partition function Z); it is typical to normalize the f_α in some way (to sum to one, or have maximum value one) after each update in order to avoid numerical issues.

The resulting algorithm is extremely intuitive: for each factor f_α in turn, we compute the current marginal probabilities $p(X_\alpha)$, and update f_α proportionally by $\hat{p}(X_\alpha)/p(X_\alpha)$. This forces the model to match the desired expectations $\hat{p}$ on the variables X_α. However, the update also alters the expectations of other cliques, so that they may no longer satisfy the zero gradient condition and may need to be re-optimized.

Example 4-20 : Iterative Proportional Fitting

Suppose that we observe a dataset of $m = 10$ observations of $X = [A, B, C]$, whose joint empirical frequency is shown in the table at right. We wish to fit a pairwise graphical model, i.e.,

$$p(X) \propto f(X) = f_{AB}(A, B)\, f_{BC}(B, C)\, f_{AC}(A, C).$$

Starting with an arbitrary initialization of the f_α (for example, $f_\alpha(X_\alpha) \equiv 1$, so that $p(X)$ is the uniform distribution), we can iteratively perform IPF updates on each f_α.

ABC	$\hat{p}(A,B,C)$
000	0.2
001	0.2
010	0.2
011	0.1
100	0.0
101	0.1
110	0.1
111	0.1

For the purposes of illustration, we start with a non-uniform initialization, and compute the current marginal $p(A, B)$:

AB	f_{AB}
0 0	0.1
0 1	0.2
1 0	0.3
1 1	0.4

AC	f_{AC}
0 0	0.2
0 1	0.6
1 0	0.4
1 1	0.8

BC	f_{BC}
0 0	0.5
0 1	0.1
1 0	0.1
1 1	0.5

$\Rightarrow$

ABC	$f(x)$	$\Rightarrow$	$p(x)$
0 0 0	0.010		0.029
0 0 1	0.006		0.017
0 1 0	0.040		0.116
0 1 1	0.016		0.047
1 0 0	0.006		0.017
1 0 1	0.090		0.262
1 1 0	0.016		0.047
1 1 1	0.160		0.465

$\Rightarrow$

AB	p_{AB}
0 0	0.047
0 1	0.163
1 0	0.279
1 1	0.512

Clearly, our initial model's marginal probabilities do not match those from the data; for example, $p_{AB}(A = 0, B = 0) = 0.047$, while $\hat{p}_{AB}(A = 0, B = 0) = 0.2 + 0.2 = 0.4$. We can update each entry of $f_{AB}(a, b)$ by multiplying by the value $\hat{p}_{AB}(a, b)/p_{AB}(a, b)$, giving:

AB	f_{AB}
0 0	0.860
0 1	0.369
1 0	0.108
1 1	0.156

AC	f_{AC}
0 0	0.2
0 1	0.6
1 0	0.4
1 1	0.8

BC	f_{BC}
0 0	0.5
0 1	0.1
1 0	0.1
1 1	0.5

$\Rightarrow$

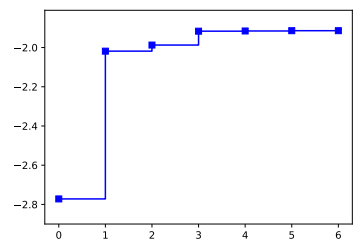
ABC	$f(x)$	$\Rightarrow$	$p(x)$
0 0 0	0.086		0.250
0 0 1	0.052		0.150
0 1 0	0.074		0.214
0 1 1	0.030		0.086
1 0 0	0.002		0.006
1 0 1	0.032		0.094
1 1 0	0.006		0.018
1 1 1	0.063		0.182

$\Rightarrow$

AB	p_{AB}
0 0	0.4
0 1	0.3
1 0	0.1
1 1	0.2

after which the updated model's marginal, p_{AB}, exactly matches the empirical marginal probabilities.

We next compute p_{BC} and update f_{BC}, and then p_{AC} to update f_{AC}. However, after these updates, our new model no longer has the correct marginal, p_{AB}; we can therefore re-compute p_{AB} to re-update f_{AB}, and repeat, cycling through each factor f_α in our model. Each update increases the log-likelihood of the model (shown at right), optimally setting the parameters of that factor given the values of the other factors. Eventually, these updates will converge, at which point *all* the marginals p_α corresponding to some factor f_α will match the empirical marginals of $\hat{p}$, e.g.:



AB	f_{AB}
0 0	0.876
0 1	0.360
1 0	0.103
1 1	0.161

AC	f_{AC}
0 0	0.169
0 1	0.702
1 0	0.493
1 1	0.644

BC	f_{BC}
0 0	0.423
0 1	0.122
1 0	0.357
1 1	0.392

$\Rightarrow$

ABC	$f(x)$	$\Rightarrow$	$p(x)$
0 0 0	0.063		0.182
0 0 1	0.075		0.218
0 1 0	0.075		0.218
0 1 1	0.028		0.082
1 0 0	0.006		0.018
1 0 1	0.028		0.082
1 1 0	0.028		0.082
1 1 1	0.041		0.118

$\Rightarrow$

AB	p_{AB}
0 0	0.4
0 1	0.3
1 0	0.1
1 1	0.2

However, it is not necessarily the case that $p(X) = \hat{p}(X)$. For example, $\hat{p}(X = [1, 0, 0]) = 0$, but $p(X)$ is strictly positive, with $p(X = [1, 0, 0]) = 0.018$. We can view a factorization assumption like that of $p(X)$ as *regularizing* our model, by restricting the set of distributions over which we search to those that satisfy the factorization. Although at first glance, our representation of $f(X)$ appears to have $3 \times 4 = 12$ parameters, many of these parameters correspond to features that are linearly dependent (see Section 4.5). In fact, a pairwise binary distribution over A, B, C has only 6 linearly independent parameters[2]. In contrast, an arbitrary distribution over A, B, C has $7 = 8 - 1$ degrees of freedom; thus not all possible distributions $\hat{p}$ can be represented as a pairwise model p.

Pseudolikelihood

As we shall see in Chapter 5, exactly evaluating the partition function or marginal probabilities of a large model can be computationally difficult. One solution is to substitute a simpler-to-evaluate objective function for the optimization, i.e., an objective that does not require a difficult normalization step.

[2]For example, in the Boltzmann parameterization, these are the binary features A, B, C, and $A \cdot B$, $B \cdot C$, $A \cdot C$.

One possibility is to use a quantity called the **pseudo-likelihood** instead, first proposed by Besag (1975). The pseudo-log-likelihood is a surrogate for the true log-likelihood function that is constructed from the conditional distributions of the variables:

$$\mathcal{PL} = \sum_s \sum_i \log p(X_i = x_i^{(s)} \mid X_{\neg i} = x_{\neg i}^{(s)})$$

where $X_{\neg i}$ are all variables in X except X_i. For context, we can contrast the pseudo-log-likelihood with the actual log-likelihood in a simple example.

Example 4-21 : Pseudo-likelihood

Let $p(A, B)$ be a probability distribution, and observe a single data point (a, b). By the chain rule, the log-likelihood is,

$$\mathcal{L} = \log p(A = a) + \log p(B = b|A = a).$$

In contrast, the pseudo-log-likelihood is,

$$\mathcal{PL} = \log p(A = a|B = b) + \log p(B = b|A = a).$$

In other words, the actual likelihood can be evaluated as the probability of $A = a$ occuring, combined with the probability that $B = b$ then happens *given* $A = a$. The first term of the the pseudo-likelihood, on the other hand, *also* gets the extra information of knowing that $B = b$ will happen when evaluating A's term, as well.

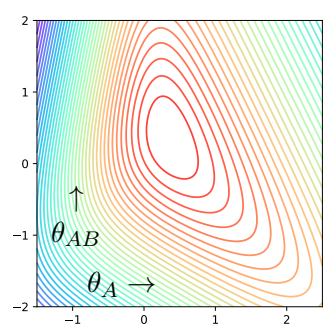
More concretely, suppose that A and B are binary, and that $p(A, B)$ is an Ising model,

$$p(A, B) \propto \exp\left(\theta_A A + \theta_B B + \theta_{AB} AB\right)$$

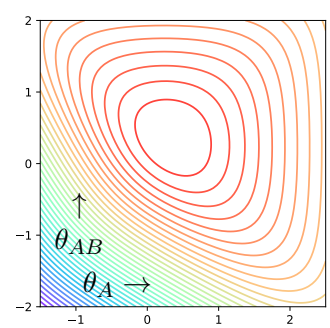
with $A, B \in \{-1, +1\}$. We observe a few data points D, shown below. If we assume that $\theta_A = \theta_B$ (in order to have only two parameters to facilitate visualization, and since the data are symmetric in A and B), we can plot the log-likelihood and pseudo-log-likelihood of our model as a function of θ_A and θ_{AB}:

A	B
-1	-1
-1	+1
+1	-1
+1	+1
+1	+1
+1	+1

Data D



Log-likelihood $\mathcal{L}$



Pseudo-log-likelihood $\mathcal{PL}$

We can see that, in this case, the maxima are quite similar, near $\theta_A = \theta_B \approx \frac{1}{3}$, $\theta_{AB} \approx \frac{1}{3}$. However, in some parts of the parameter space, the two behave fairly differently. For example, in the upper left corner we see that the likelihood falls off quickly, while the pseudo-likelihood does not. This regime corresponds to A, B being positively correlated ($\theta_{AB} > 0$), but both more likely to be negative than positive ($\theta_A = \theta_B < 0$). This is a poor match to the data, since we have more positive than negative examples in our data set, giving it low likelihood. However, *given* $B = +1$, the outcome $A = +1$ remains fairly likely, and vice versa. The pseudo-likelihood overcounts this conditioning information, penalizing these parameter values less than the true likelihood.

The pseudo-likelihood does not quite make sense as a probability, since we get the benefit of exploiting our variables' correlation twice. However, as with maximum likelihood, the maximum pseudo-likelihood estimator remains statistically consistent, and can be significantly easier to compute. Each conditional distribution is a function only of that variable's Markov blanket, and is easy to normalize since it is a function of only a single variable.

Example 4-22 : Boltzmann Pseudo-likelihood

For the Boltzmann machine model of Example 4-4, we can see that the conditional distribution of each X_i is given by,

$$p(X_i|X_{\neg i} = x_{\neg i}) \propto \exp\Big(X_i\big(\sum_{j\neq i}\theta_{ij}x_j + b_i\big)\Big)$$

Since X_i has only two values ($\{0, 1\}$), we have

$$p(X_i = 1|X_{\neg i} = x_{\neg i}) = \Big(1 + \exp\big(-b_i - \sum_{j\neq i}\theta_{ij}x_j\big)\Big)^{-1},$$

i.e., each $p(X_i|x_{\neg i})$ is a logistic regression-like likelihood term. Note, however, that each θ_{ij} appears in two terms: $p(X_i|X_{\neg i})$ and $p(X_j|X_{\neg j})$. We can thus view optimizing the pseudo-likelihood $\mathcal{PL}$ as jointly training n coupled logistic regression models.

Example / simulation? Note about if model is misspecified?
Somewhere? phase transitions in 3-SAT problems? NP-completeness?